

# 字符串

Morning Flower and Evening Oath

XCPC vector 模板分册

# 目录

|                          |           |
|--------------------------|-----------|
| 使用约定                     | ii        |
| <b>第一章 前缀函数与 KMP 算法</b>  | <b>1</b>  |
| 1.1 前缀函数                 | 1         |
| 1.2 KMP 匹配               | 1         |
| <b>第二章 Z 函数 (扩展 KMP)</b> | <b>2</b>  |
| 2.1 Z 函数                 | 2         |
| <b>第三章 AC 自动机</b>        | <b>3</b>  |
| 3.1 AC 自动机出现次数           | 3         |
| <b>第四章 后缀数组 (SA)</b>     | <b>5</b>  |
| 4.1 后缀数组简介               | 5         |
| 4.1.1 倍增后缀数组与 LCP        | 5         |
| 4.1.2 后缀 LCP 与子串比较       | 7         |
| 4.1.3 两个前缀的最长公共后缀        | 8         |
| 4.1.4 重复子串 AA 的起止位置计数    | 8         |
| <b>第五章 后缀自动机 (SAM)</b>   | <b>9</b>  |
| 5.1 后缀自动机                | 9         |
| <b>第六章 Manacher</b>      | <b>11</b> |
| 6.1 Manacher 回文半径        | 11        |
| <b>第七章 回文树</b>           | <b>12</b> |
| 7.1 回文树 PAM: 回文后缀与出现次数   | 12        |
| <b>第八章 最小表示法</b>         | <b>14</b> |
| 8.1 最小循环表示               | 14        |

# 使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

# 第一章 前缀函数与 KMP 算法

## 1.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j]) j = p[j - 1];
9         p[i] = j + (s[i] == s[j]);
10    }
11    return p;
12 }
```

## 1.2 KMP 匹配

`kmp_match(s,t)` 返回非空模式串 `t` 在 `s` 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j]) j = p[j - 1];
9         j += (s[i] == t[j]);
10        if (j == (int)t.size())
11        {
12            ans.push_back(i - j + 1);
13            j = p[j - 1];
14        }
15    }
16    return ans;
17 }
```

## 第二章 Z 函数 (扩展 KMP)

### 2.1 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP, 约定 `z[0]=n`, 空串返回空数组。O(n)。

```
1 vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n) a[0] = n;
6     for (int i = 1, l = 0, r = 0; i < n; i++)
7     {
8         if (i < r) a[i] = min(r - i, a[i - l]);
9         while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
10        if (i + a[i] > r)
11        {
12            l = i;
13            r = i + a[i];
14        }
15    }
16    return a;
17 }
```

# 第三章 AC 自动机

## 3.1 AC 自动机出现次数

字母表  $a..z$ ,  $\text{add}(s)$  返回终止状态, 要求模式非空; 全部插入后  $\text{build}()$ , 只允许一次构建。 $\text{count}(\text{text})$  返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建  $O(26N)$ , 查询  $O(|\text{text}|+N)$ 。

```
1 struct AhoCorasick
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;
7     };
8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    int add(const string &s)
14    {
15        assert(!built && !s.empty());
16        int u = 0;
17        for (char ch : s)
18        {
19            int c = ch - 'a';
20            assert(0 <= c && c < 26);
21            if (!a[u].go[c])
22            {
23                int v = (int)a.size();
24                a[u].go[c] = v;
25                a.push_back(Node{});
26            }
27            u = a[u].go[c];
28        }
29        return u;
30    }
31
32    void build()
33    {
34        assert(!built);
35        built = true;
36        queue<int> q;
37        for (int v : a[0].go)
38            if (v) q.push(v);
39        while (!q.empty())
40        {
41            int u = q.front();
42            q.pop();
43            order.push_back(u);
44            for (int c = 0; c < 26; c++)
45            {
```

```
46         int v = a[u].go[c];
47         if (v)
48         {
49             a[v].fail = a[a[u].fail].go[c];
50             q.push(v);
51         }
52         else
53             a[u].go[c] = a[a[u].fail].go[c];
54     }
55 }
56
57
58 vector<long long> count(const string &s) const
59 {
60     assert(built);
61     vector<long long> ans(a.size());
62     int u = 0;
63     for (char ch : s)
64     {
65         u = a[u].go[ch - 'a'];
66         ++ans[u];
67     }
68     for (int i = (int)order.size() - 1; i >= 0; i--)
69         ans[a[order[i]].fail] += ans[order[i]];
70     return ans;
71 }
72 };
```

# 第四章 后缀数组 (SA)

## 4.1 后缀数组简介

### 4.1.1 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串, 按 unsigned char 的 0..255 数值比较, 包含内嵌零字节; 也可构造 SuffixArray(a,alphabet), 整数序列要求  $0 \leq a[i] < \text{alphabet}$ ,  $1 \leq \text{alphabet} < \text{INT\_MAX}$ 。init(a,alphabet) 可重复重建。长度须小于 INT\_MAX。sa[k] 为第 k 个非空后缀起点, rk[i] 为排名, 全部 0-based; lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀, lcp[0]=0。空序列合法, 三数组均为空。无需手动追加哨兵, 也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序, 倍增  $O(n \log n + \text{alphabet})$ , 空间  $O(n + \text{alphabet})$ ; 字节串 alphabet=256。大整数先自行保序离散化, 避免按最大原值开桶。Kasai 求 lcp 为  $O(n)$ 。不同起点 x,y 的 LCP 为  $\text{lcp}[\min(\text{rk}[x], \text{rk}[y]) + 1.. \max(\text{rk}[x], \text{rk}[y])]$  的最小值; 本结构不内建 RMQ, 需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造, 不替代 DC3/SA-IS。

```
1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for (unsigned char c : s) a.push_back(c);
10        init(a, 256);
11    }
12
13    SuffixArray(const vector<int> &s, int alphabet) { init(s, alphabet); }
14
15    void init(const vector<int> &s, int alphabet)
16    {
17        assert(s.size() < INT_MAX);
18        assert(1 <= alphabet && alphabet < INT_MAX);
19        int n = (int)s.size();
20        sa.resize(n);
21        rk.resize(n);
22        lcp.assign(n, 0);
23        if (!n) return;
24        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
25        int m = alphabet;
26        for (int i = 0; i < n; i++)
27        {
28            assert(0 <= s[i] && s[i] < alphabet);
29            rk[i] = s[i] + 1;
30            cnt[rk[i]]++;
31        }
32        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
33        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```

34     for (int k = 1; k < n; k *= 2)
35     {
36         int p = 0;
37         for (int i = n - k; i < n; i++) order[p++] = i;
38         for (int i : sa)
39             if (i >= k) order[p++] = i - k;
40         fill(cnt.begin(), cnt.begin() + m + 1, 0);
41         for (int x : rk) cnt[x]++;
42         for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
43         for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
44         old.swap(rk);
45         auto second = [&](int i)
46         {
47             return i < n - k ? old[i + k] : 0;
48         };
49         m = 1;
50         rk[sa[0]] = m;
51         for (int i = 1; i < n; i++)
52         {
53             int a = sa[i - 1], b = sa[i];
54             if (old[a] != old[b] || second(a) != second(b)) m++;
55             rk[b] = m;
56         }
57         if (m == n) break;
58     }
59     for (int i = 0; i < n; i++) rk[sa[i]] = i;
60     for (int i = 0, k = 0; i < n; i++)
61     {
62         if (!rk[i])
63         {
64             k = 0;
65             continue;
66         }
67         int j = sa[rk[i] - 1];
68         while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
69         lcp[rk[i]] = k;
70         if (k) --k;
71     }
72 }
73 };

```

### 4.1.2 后缀 LCP 与子串比较

先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n] 与 s[y..n] 的最长公共前缀， $0 \leq x, y \leq n$ ，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表  $O(n \log n)$  时间、空间，每次查询  $O(1)$ ，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。

```

1 struct SuffixLCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>> st;
6
7     SuffixLCP(const SuffixArray &suffix)
8     {
9         n = suffix.sa.size();
10        rk = suffix.rk;
11        lg.assign(n + 1, 0);
12        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
13        st.assign(lg[n] + 1, {});
14        st[0] = suffix.lcp;
15        for (int k = 1; k <= lg[n]; k++)
16        {
17            int len = 1 << k;
18            st[k].resize(n - len + 1);
19            for (int i = 0; i <= n - len; i++)
20                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
21        }
22    }
23
24    int query(int x, int y) const
25    {
26        assert(0 <= x && x <= n && 0 <= y && y <= n);
27        if (x == y || x == n || y == n) return n - max(x, y);
28        int a = rk[x], b = rk[y];
29        if (a > b) swap(a, b);
30        int k = lg[b - a];
31        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
32    }
33
34    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
35    int compare(int l1, int r1, int l2, int r2) const
36    {
37        assert(0 <= l1 && l1 <= r1 && r1 <= n);
38        assert(0 <= l2 && l2 <= r2 && r2 <= n);
39        int a = r1 - l1, b = r2 - l2;
40        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
41        return rk[l1] < rk[l2] ? -1 : 1;
42    }
43 };

```

### 4.1.3 两个前缀的最长公共后缀

`prefix_lcs(reversed,x,y)` 返回原序列 `s[0..x)` 与 `s[0..y)` 的最长公共后缀，端点满足  $0 \leq x, y \leq n$ 。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 `reversed.query(n-x,n-y)`，单次  $O(1)$ 。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```
1 // reversed must be built on the reversed original sequence.
2 int prefix_lcs(const SuffixLCP &reversed, int x, int y)
3 {
4     assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5     return reversed.query(reversed.n - x, reversed.n - y);
6 }
```

### 4.1.4 重复子串 AA 的起止位置计数

`square_counts(forward,reversed)` 返回 `(start,finish)` 两个长度 `n` 的 long long 数组，`start[i]/finish[i]` 为开始/结束于位置 `i` 的非空 AA 型子串个数；结束位置是最后一个字符下标，均 0-based。两对象必须分别对应同一原序列及其反序列，长度相同，允许空序列。枚举半长 `len`，并以其整数倍 `i` 为锚点；向左匹配截断为 `len-1`，向右截断为 `len`，由 LCP/LCS 得出合法起点连续区间，差分累计。左侧不能放宽到 `len`，否则相邻锚点会重复计数。预处理查询器后，此函数  $O(n \log n)$  时间、 $O(n)$  额外空间。不同出现位置、不同半长分别计数；不要求 `A` 为最小周期。P1117 的 AABB 拆分数为所有分界点 `finish[i]*start[i+1]` 之和，累加类型需按题目范围选择，函数本身只返回起止计数。

```
1 // Count nonempty squares AA by their starting and ending positions.
2 pair<vector<long long>, vector<long long>> square_counts(const SuffixLCP &forward,
3                                                         const SuffixLCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for (int len = 1; len <= n / 2; len++)
9         for (int i = 0; i < n - len; i += len)
10            {
11                int j = i + len;
12                int left = min(len - 1, prefix_lcs(reversed, i, j));
13                int right = min(len, forward.query(i, j));
14                if (left + right < len) continue;
15                int l = i - left, r = i + right - len;
16                start[l]++;
17                start[r + 1]--;
18                finish[l + 2 * len - 1]++;
19                finish[r + 2 * len]--;
20            }
21     for (int i = 1; i < n; i++)
22     {
23         start[i] += start[i - 1];
24         finish[i] += finish[i - 1];
25     }
26     start.resize(n);
27     finish.resize(n);
28     return {start, finish};
29 }
```

# 第五章 后缀自动机 (SAM)

## 5.1 后缀自动机

extend(c) 接收整数 0..25; distinct() 返回不同非空子串数, counts() 返回 endpos 次数且不修改结构, 可重复调用。状态 a, 初始状态 0, link 根为 -1; 空间  $O(26n)$ , 构建  $O(26n)$ 。

```
1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while (p != -1 && !a[p].go[c])
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if (p == -1)
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if (a[p].len + 1 == a[q].len)
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while (p != -1 && a[p].go[c] == q)
41                {
42                    a[p].go[c] = clone;
43                    p = a[p].link;
44                }
45                a[q].link = a[cur].link = clone;
```

```

46         }
47     }
48     last = cur;
49 }
50
51 vector<long long> counts() const
52 {
53     vector<int> bucket(a[last].len + 1, order(a.size()));
54     vector<long long> ans(a.size());
55     for (auto &v : a) ++bucket[v.len];
56     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
57     for (int i = 0; i < (int)a.size(); i++)
58     {
59         order[--bucket[a[i].len]] = i;
60         ans[i] = a[i].occ;
61     }
62     for (int i = (int)order.size() - 1; i > 0; i--)
63         ans[a[order[i]].link] += ans[order[i]];
64     return ans;
65 }
66
67 long long distinct() const
68 {
69     long long ans = 0;
70     for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
71     return ans;
72 }
73 };

```

# 第六章 Manacher

## 6.1 Manacher 回文半径

manacher(s) 返回 odd 与 even; odd[i] 的半径包含中心, even[i] 的中心在 i 前。字符串及下标为 0-based, 空串合法,  $O(n)$ 。

```
1 // odd[i] includes the center; even[i] is centered before i.
2 pair<vector<int>, vector<int>> manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for (int i = 0, l = 0, r = -1; i < n; i++)
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
10        odd[i] = k--;
11        if (i + k > r)
12        {
13            l = i - k;
14            r = i + k;
15        }
16    }
17    for (int i = 0, l = 0, r = -1; i < n; i++)
18    {
19        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
20        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
21        even[i] = k--;
22        if (i + k > r)
23        {
24            l = i - k - 1;
25            r = i + k;
26        }
27    }
28    return {odd, even};
29 }
```

# 第七章 回文树

## 7.1 回文树 PAM: 回文后缀与出现次数

`add(ch)` 追加小写英文字母，返回最长回文后缀节点，亦保存于 `last`。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；`distinct()` 为不同非空回文串数。`t[u].len` 为长度，`fail` 为最长真回文后缀链接，`suffixes` 为该节点的非空回文后缀数量；追加后的 `t[last].suffixes` 就是本位置结尾的回文串数。`total` 为当前全部回文子串的出现次数之和。`end` 为首次出现时的 0-based 右端点，可恢复原串子段；`hits` 仅统计作为最长后缀的次数，不能当出现次数。`occurrences()` 返回按节点编号的完整出现次数副本，根置零， $O(n)$ ；重复调用以及查询后继续追加均安全。重新赋值 `PalindromicTree()` 清空重建。固定 26 字符集下建树  $O(n)$ ，空间  $O(26n)$ 。

```
1 struct PalindromicTree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    vector<Node> t;
11    vector<int> s;
12    int last = 0;
13    long long total = 0;
14
15    PalindromicTree() : t(2), s(1, -1)
16    {
17        t[0].fail = 1;
18        t[1].len = -1;
19    }
20
21    int get_fail(int u) const
22    {
23        int pos = s.size() - 1;
24        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
25        return u;
26    }
27
28    // Append a lowercase letter; return the longest palindromic suffix node.
29    int add(char ch)
30    {
31        int c = ch - 'a';
32        assert(0 <= c && c < 26);
33        s.push_back(c);
34        int u = get_fail(last);
35        if (!t[u].next[c])
36        {
37            int v = t.size();
38            t.push_back(Node{});
39            t[v].len = t[u].len + 2;
40            t[v].fail = t[get_fail(t[u].fail)].next[c];
41            t[v].suffixes = t[t[v].fail].suffixes + 1;
```

```
42         t[v].end = (int)s.size() - 2;
43         t[u].next[c] = v;
44     }
45     last = t[u].next[c];
46     t[last].hits++;
47     total += t[last].suffixes;
48     return last;
49 }
50
51 int distinct() const { return (int)t.size() - 2; }
52
53 // Copy-and-propagate: repeated calls and later appends remain valid.
54 vector<long long> occurrences() const
55 {
56     vector<long long> count(t.size());
57     for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
58     for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
59     count[0] = 0;
60     count[1] = 0;
61     return count;
62 }
63 };
```

# 第八章 最小表示法

## 8.1 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。O(n)。

```
1 int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n) return 0;
5     while (i < n && j < n && k < n)
6     {
7         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
8         if (a == b)
9         {
10             ++k;
11             continue;
12         }
13         if (a > b)
14         {
15             i += k + 1;
16             if (i == j) ++i;
17         }
18         else
19         {
20             j += k + 1;
21             if (i == j) ++j;
22         }
23         k = 0;
24     }
25     return min(i, j);
26 }
```

# 算法索引

AhoCorasick, 3

kmp\_match, 1

manacher, 11

minimum\_rotation, 14

PalindromicTree, 12

prefix\_function, 1

prefix\_lcs, 8

square\_counts, 8

SuffixArray, 5

SuffixAutomaton, 9

SuffixLCP, 7

z\_function, 2